

Reinforcement Learning: From Expected Returns to Extreme Risks

Edinburgh Summer School on Generative AI for Extremes

Giorgia Ramponi

University of Zurich

8–11 September 2026
JCMB, Edinburgh



THE UNIVERSITY
of EDINBURGH



A forecast is not yet a decision

A city has one movable flood barrier.

Closing it costs £1M.

Leaving it open during a flood costs £100M.

The model predicts a 2% flood probability.

What should we do?

Close now?

Wait for another observation?

Change the warning threshold?

The answer depends on future information, repeated decisions, and our attitude to risk.

Prediction and control answer different questions

Supervised / generative learning

What may happen?

$$x \mapsto \hat{y} \quad \text{or} \quad x \sim p_{\theta}(x)$$

Fit a distribution from data.

Reinforcement learning

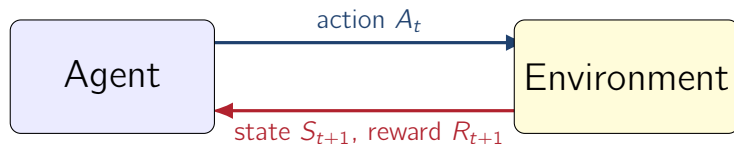
What should an agent do?

$$s \mapsto a \mapsto s'$$

Optimize consequences over time.

RL turns models of uncertainty into **sequential decisions**.

Reinforcement learning is learning by interaction



- Feedback is evaluative: a scalar reward, not the correct action.
- Actions change which data the agent will observe next.
- The goal is long-term performance, not immediate reward alone.

Sutton and Barto, *Reinforcement Learning*, 2nd ed., 2018.

The Markov decision process is the basic model

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, (H))$$

$S_t \in \mathcal{S}$ information used for the decision

$A_t \in \mathcal{A}$ intervention chosen by the agent

$P(s' | s, a)$ how the system evolves

$r(s, a, s')$ immediate preference or cost

$\gamma \in [0, 1)$ how strongly future consequences matter

H the horizon, the number of interactions with the environment
(could be ∞)

Markov means: the state at time step t contains everything needed to predict the next step $t + 1$.

A policy commits actions to states

Deterministic policy

$$a = \pi(s)$$

One action for each state.

Stochastic policy

$$a \sim \pi(\cdot | s)$$

A distribution over actions.

What is the state in the flood control problem?

A policy commits actions to states

Deterministic policy

$$a = \pi(s)$$

One action for each state.

Stochastic policy

$$a \sim \pi(\cdot | s)$$

A distribution over actions.

What is the state in the flood control problem? Examples of a state in flood control:

- reservoir level, soil moisture, forecast ensemble;
- barrier status and time remaining before landfall;
- uncertainty summaries and operational constraints.

A policy commits actions to states

Deterministic policy

$$a = \pi(s)$$

One action for each state.

Stochastic policy

$$a \sim \pi(\cdot | s)$$

A distribution over actions.

What are the actions in the flood control problem?

A policy commits actions to states

Deterministic policy

$$a = \pi(s)$$

One action for each state.

Stochastic policy

$$a \sim \pi(\cdot | s)$$

A distribution over actions.

What are the actions in the flood control problem? Examples of actions in flood control:

- release or retain water;
- open or close the flood barrier;
- wait for another observation.

The objective aggregates consequences over time

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$\pi^* \in \arg \max_{\pi} J(\pi), \quad J(\pi) = \mathbb{E}_{\pi}[G_0]$$

- γ trades off near-term and distant outcomes.
- The expectation averages across trajectories generated by π .
- Reward design specifies the task (and can specify it badly.)

Values answer “what happens if I continue this way?”

$$V_t^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s],$$

$$Q_t^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

State value V^π

How desirable is this situation under policy π ?

Action value Q^π

How desirable is taking action a now, then following π ?

The Bellman equation makes a long horizon local

$$V_t^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma V_{t+1}^\pi(S_{t+1}) \mid S_t = s]$$



Idea: update an estimate using another estimate.

Bellman, *Dynamic Programming*, 1957.

If the model is known, we can plan

Optimality backup

$$V^*(s) = \max_a \left[\sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V^*(s')] \right]$$

- 1 Start from arbitrary value estimates.
- 2 Repeatedly apply the Bellman optimality backup.
- 3 Choose actions greedily from the converged values.

If the model is known, we can plan

Optimality backup

$$V^*(s) = \max_a \left[\sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V^*(s')] \right]$$

- 1 Start from arbitrary value estimates.
- 2 Repeatedly apply the Bellman optimality backup.
- 3 Choose actions greedily from the converged values.

Dynamic programming is powerful—but requires P and manageable state spaces.

Value iteration repeatedly applies the optimality backup

Value update

Starting from an arbitrary value function V_0 , compute

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')]$$

for every state s .

- 1 Initialize $V_0(s)$, for example with $V_0(s) = 0$.
- 2 Apply the update until the values change negligibly:

$$\|V_{k+1} - V_k\|_{\infty} < \epsilon.$$

- 3 Extract the greedy policy:

$$\pi(s) \in \arg \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')].$$

Value iteration repeatedly applies the optimality backup

Value update

Starting from an arbitrary value function V_0 , compute

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')]$$

for every state s .

- 1 Initialize $V_0(s)$, for example with $V_0(s) = 0$.
- 2 Apply the update until the values change negligibly:

$$\|V_{k+1} - V_k\|_{\infty} < \epsilon.$$

- 3 Extract the greedy policy:

$$\pi(s) \in \arg \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')].$$

For a finite discounted MDP, V_k converges to the optimal value function V^* .

Without a model, temporal-difference learning uses experience

One-step TD update

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD error } \delta_t}$$

- Learn after every transition; no full episode required.
- The TD error is new information relative to the current prediction.
- Sampling replaces an explicit transition model.

Q-learning learns control directly

Off-policy update

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

- Behavior can explore; the update targets the greedy policy.
- After learning: $\pi(s) \in \arg \max_a Q(s, a)$.
- In a finite tabular MDP, sufficient exploration and suitable step sizes yield convergence.

Watkins and Dayan, "Q-learning," *Machine Learning*, 1992.

The Q-learning algorithm

Initialize

Set $Q(s, a)$ arbitrarily for all state–action pairs (and $Q(s, a) = 0$ for terminal states).

- ❶ For each episode, initialize the state S .
- ❷ While S is not terminal:
 - ❶ Choose A from S using an exploratory policy, such as ϵ -greedy with respect to Q .
 - ❷ Execute A and observe reward R and next state S' .
 - ❸ Update

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right].$$

- ❹ Set $S \leftarrow S'$.

The Q-learning algorithm

Initialize

Set $Q(s, a)$ arbitrarily for all state–action pairs (and $Q(s, a) = 0$ for terminal states).

- ❶ For each episode, initialize the state S .
- ❷ While S is not terminal:
 - ❶ Choose A from S using an exploratory policy, such as ϵ -greedy with respect to Q .
 - ❷ Execute A and observe reward R and next state S' .
 - ❸ Update

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right].$$

- ❹ Set $S \leftarrow S'$.

Output: $\pi(s) \in \arg \max_a Q(s, a)$.

Pure greedy can stop exploration

Consider a single state with two available actions:

Action	True reward	Initial estimate
a_1	1	$Q(s, a_1) = 0$
a_2	2	$Q(s, a_2) = 0$

Pure greedy can stop exploration

Consider a single state with two available actions:

Action	True reward	Initial estimate
a_1	1	$Q(s, a_1) = 0$
a_2	2	$Q(s, a_2) = 0$

Suppose ties are broken in favour of a_1 :

$$a_1 \text{ is selected} \implies Q(s, a_1) \leftarrow 1.$$

Pure greedy can stop exploration

Consider a single state with two available actions:

Action	True reward	Initial estimate
a_1	1	$Q(s, a_1) = 0$
a_2	2	$Q(s, a_2) = 0$

Suppose ties are broken in favour of a_1 :

$$a_1 \text{ is selected} \implies Q(s, a_1) \leftarrow 1.$$

A purely greedy agent now always selects

$$\arg \max_a Q(s, a) = a_1,$$

because $Q(s, a_1) = 1 > Q(s, a_2) = 0$.

The problem

The agent never tries a_2 , so it never discovers that a_2 is better.

ϵ -greedy keeps learning possible

At each step, choose

$$A_t = \begin{cases} \arg \max_a Q(S_t, a), & \text{with probability } 1 - \epsilon, \\ \text{a uniformly random action,} & \text{with probability } \epsilon. \end{cases}$$

Most of the time: exploit

Select the action currently believed to be best.

Occasionally: explore

Try other actions and improve the value estimates.

ϵ -greedy keeps learning possible

At each step, choose

$$A_t = \begin{cases} \arg \max_a Q(S_t, a), & \text{with probability } 1 - \epsilon, \\ \text{a uniformly random action,} & \text{with probability } \epsilon. \end{cases}$$

Most of the time: exploit

Select the action currently believed to be best.

Occasionally: explore

Try other actions and improve the value estimates.

In the previous example, exploration eventually selects a_2 :

$$Q(s, a_2) \leftarrow 2 \implies \arg \max_a Q(s, a) = a_2.$$

ϵ -greedy prevents early estimates from becoming permanent decisions.

Exploration is data collection under intervention

Exploit

Choose the action currently believed to be best.

$$A_t = \arg \max_a Q(S_t, a)$$

Explore

Choose informative actions that may improve future decisions.

e.g. ϵ -greedy

In safety-critical systems, naive exploration may be unacceptable.

Deep RL replaces tables with function approximation

$$Q(s, a; \theta) \approx Q^*(s, a)$$

A neural network can generalize across high-dimensional states such as maps, images, or forecast fields.

DQN tricks

- replay buffer
- target network
- clipped rewards / errors

Approximation enables scale, but introduces instability and extrapolation error.

Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 2015.

Policy gradients optimize the policy itself

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t]$$

- Works naturally with continuous or stochastic actions.
- Increases probability of actions followed by high return.
- Monte Carlo estimates can have high variance.

Policy gradients optimize the policy itself

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t]$$

- Works naturally with continuous or stochastic actions.
- Increases probability of actions followed by high return.
- Monte Carlo estimates can have high variance.

Can we replace the noisy return G_t with a better learning signal?

Williams, "Simple statistical gradient-following algorithms for connectionist RL," 1992.

A critic provides a less noisy learning signal

REINFORCE weights each policy update using the complete return G_t :

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t] .$$

A critic provides a less noisy learning signal

REINFORCE weights each policy update using the complete return G_t :

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t] .$$

We can subtract a state-dependent baseline without changing the expected gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - V_w(S_t))] .$$

Advantage estimate

$$\hat{A}_t = G_t - V_w(S_t)$$

measures whether action A_t produced a better outcome than expected in state S_t .

A critic provides a less noisy learning signal

REINFORCE weights each policy update using the complete return G_t :

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t] .$$

We can subtract a state-dependent baseline without changing the expected gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - V_w(S_t))] .$$

Advantage estimate

$$\hat{A}_t = G_t - V_w(S_t)$$

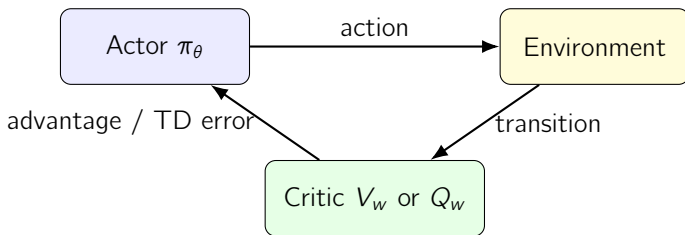
measures whether action A_t produced a better outcome than expected in state S_t .

Instead of waiting for the complete return, we can use the TD error:

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

This creates two learners: a policy actor and a value critic.

Actor-critic combines decisions with evaluation



- The actor updates the policy using the advantage or TD error.
- The critic learns a value function that evaluates the actor's decisions.
- PPO is commonly implemented in actor-critic form and limits excessively large policy updates.

Schulman et al., "Proximal Policy Optimization Algorithms," 2017.

Checkpoint: identify the learning problem

A reservoir operator chooses daily releases

State: storage, forecasts, demand.

Action: release volume.

Reward: supply benefit minus flood and shortage costs.

- 1 What information is missing from the state?
- 2 Which consequences are delayed?
- 3 Can the system safely explore online?
- 4 Is maximizing average reward really the right objective?

Expected return can hide catastrophic outcomes

$$\max_{\pi} \mathbb{E}_{\pi}[G]$$

- The expectation summarizes an entire return distribution with one number.
- Two policies can have the same mean but radically different lower tails.
- Rare catastrophes may barely affect a finite training sample.

For extreme events, we must ask not only “How well do we do on average?” but also “How bad can the bad outcomes be?”

The same expected return can conceal very different risks

Policy A: steady

Outcome	Prob.	Return
Steady	100%	10

Policy B: fragile

Outcome	Prob.	Return
Normal	99%	20
Disaster	1%	-980

Both policies have expected return 10, but Policy B concentrates harm in its lower tail.

The same expected return can conceal very different risks

Policy A: steady

Outcome	Prob.	Return
Steady	100%	10

Policy B: fragile

Outcome	Prob.	Return
Normal	99%	20
Disaster	1%	-980

Both policies have expected return 10, but Policy B concentrates harm in its lower tail.

A risk-sensitive objective must distinguish these policies.

Risk-sensitive objectives protect the downside

Let $G = \sum_{t=0}^T \gamma^t R_{t+1}$ denote the cumulative discounted return.

Three possible objectives

Expected return: $\max_{\pi} \mathbb{E}_{\pi}[G],$

Lower-tail CVaR: $\max_{\pi} \rho_{\text{CVaR}}(G; \alpha),$

Mean-semideviation: $\max_{\pi} \rho_{\text{MSD}}(G; \kappa).$

- CVaR focuses directly on the worst α fraction of returns.
- Mean-semideviation penalizes all returns that fall below the mean.

We now use CVaR because it directly targets rare, severe outcomes.

Lower-tail CVaR measures performance in the worst cases

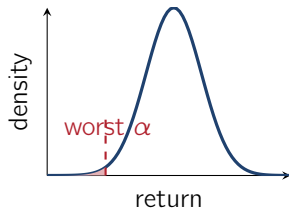
Let q_α be the α -quantile of the return:

$$q_\alpha = \inf \{g : \mathbb{P}_\pi(G \leq g) \geq \alpha\}.$$

For a continuous return distribution,

$$\rho_{\text{CVaR}}(G; \alpha) = \mathbb{E}_\pi[G \mid G \leq q_\alpha].$$

For example, $\alpha = 0.05$ means the average return among the worst 5% of trajectories.



Maximizing CVaR improves the part of the return distribution we most want to protect.

Tamar et al., "Policy Gradient for Coherent Risk Measures," NeurIPS 2015, Sec. 4.1.

A variational form makes CVaR optimizable

The lower-tail CVaR of a return can be written as

$$\rho_{\text{CVaR}}(G; \alpha) = \sup_{\nu \in \mathbb{R}} \left\{ \nu - \frac{1}{\alpha} \mathbb{E}_{\pi}[(\nu - G)_+] \right\}.$$

- At the optimum, ν is an α -quantile of G .
- $(\nu - G)_+$ is positive only when the return falls below ν .
- The objective therefore ignores good trajectories and penalizes lower-tail shortfalls.

A variational form makes CVaR optimizable

The lower-tail CVaR of a return can be written as

$$\rho_{\text{CVaR}}(G; \alpha) = \sup_{\nu \in \mathbb{R}} \left\{ \nu - \frac{1}{\alpha} \mathbb{E}_{\pi}[(\nu - G)_+] \right\}.$$

- At the optimum, ν is an α -quantile of G .
- $(\nu - G)_+$ is positive only when the return falls below ν .
- The objective therefore ignores good trajectories and penalizes lower-tail shortfalls.

CVaR turns “avoid catastrophic returns” into a differentiable policy objective.

Rockafellar and Uryasev, “Optimization of Conditional Value-at-Risk,” 2000.

The CVaR gradient learns only from lower-tail trajectories

For a trajectory τ generated by π_θ ,

$$\nabla_\theta \log P_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t | S_t).$$

Tamar et al.'s CVaR gradient becomes, in the reward convention,

$$\nabla_\theta \rho_{\text{CVaR}}(G; \alpha) = \mathbb{E}_\pi[\nabla_\theta \log P_\theta(\tau)(G - q_\alpha) \mid G \leq q_\alpha].$$

- Only trajectories in the worst α tail contribute.
- $G - q_\alpha \leq 0$ measures how far a return falls below the quantile.
- Gradient ascent reduces the probability of actions associated with severe shortfalls.

The standard policy-gradient idea survives; the risk measure changes which trajectories provide the learning signal.

Mean-semideviation protects below-mean returns

Define the lower semideviation

$$\text{SD}_-(G) = \sqrt{\mathbb{E}_\pi \left[(\mathbb{E}_\pi[G] - G)_+^2 \right]}.$$

The risk-sensitive return is

$$\rho_{\text{MSD}}(G; \kappa) = \mathbb{E}_\pi[G] - \kappa \text{SD}_-(G), \quad \kappa \in [0, 1].$$

CVaR

Focuses on a specified worst tail, such as the lowest 5%.

Mean-semideviation

Penalizes the magnitude of every return below the mean.

The appropriate risk measure depends on which failures the application must distinguish.

Reward-form equivalent of Tamar et al., NeurIPS 2015, Sec. 4.2.

Tail objectives do not replace hard safety constraints

A constrained MDP separates performance from requirements:

$$\max_{\pi} J_R(\pi) \quad \text{s.t.} \quad J_{C_i}(\pi) \leq d_i, \quad i = 1, \dots, m.$$

Optimize

water supply
renewable energy
service quality

Constrain

overflow probability
ecological minimum flow
actuator limits

Tail objectives do not replace hard safety constraints

A constrained MDP separates performance from requirements:

$$\max_{\pi} J_R(\pi) \quad \text{s.t.} \quad J_{C_i}(\pi) \leq d_i, \quad i = 1, \dots, m.$$

Optimize

water supply
renewable energy
service quality

Constrain

overflow probability
ecological minimum flow
actuator limits

CVaR expresses risk preference; constraints express requirements that should not be traded away.

Altman, *Constrained Markov Decision Processes*, 1999.

Rare events provide very little learning signal

If a catastrophe occurs independently with probability p , then observing at least one requires

$$n \geq \frac{\log(0.05)}{\log(1-p)} \approx \frac{3}{p}$$

episodes for 95% probability of seeing it.

Event probability p	Return period	Episodes for 95% chance
10^{-2}	1 in 100	299
10^{-3}	1 in 1,000	2,995
10^{-4}	1 in 10,000	29,956

A tail-aware objective cannot learn what the data never reveal.

Rare failures require a different data pipeline

- ① **Simulate deliberately:** generate more trajectories from hazardous regimes.
- ② **Correct the sampling bias:** use likelihood ratios or importance weights.
- ③ **Stress test:** evaluate under plausible shifts and worst-case models.
- ④ **Learn from existing data:** initialize the policy offline before deployment.
- ⑤ **Retain a safety layer:** enforce known physical limits during execution.

Reliable tail learning needs both a risk-aware objective and risk-relevant data.

Offline RL avoids exploration but creates a support problem

Why it helps

Learn from historical trajectories

$$\mathcal{D} = \{(s, a, r, s')\}$$

without risky online exploration.

What can fail

The learned policy selects actions that are rare or absent in $\mathcal{D}$.

Value estimates can become confidently wrong outside the data distribution.

Offline RL avoids exploration but creates a support problem

Why it helps

Learn from historical trajectories

$$\mathcal{D} = \{(s, a, r, s')\}$$

without risky online exploration.

What can fail

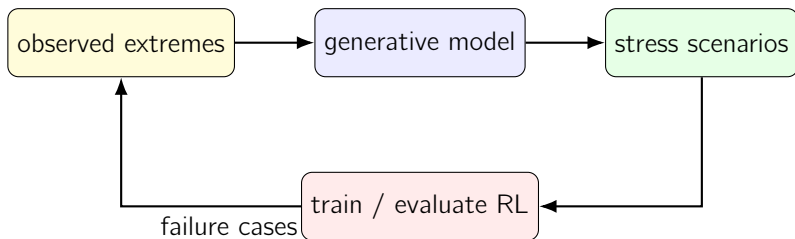
The learned policy selects actions that are rare or absent in $\mathcal{D}$.

Value estimates can become confidently wrong outside the data distribution.

Conservative offline RL trades estimated performance for support awareness.

Levine et al., "Offline Reinforcement Learning: Tutorial, Review, and Perspectives," 2020.

Generative models can expand the tail—but not certify it



- Validate tail frequencies and dependence, not only average sample quality.
- Preserve likelihood information when sampling corrections are required.
- Evaluate across several plausible generators and shifted environments.

Generated extremes are scenarios for learning and stress testing, not guarantees about the real tail.

Reliable decisions under extremes require three layers

- 1 **Objective:** specify which outcomes matter through expected return, CVaR, semideviation, and constraints.
- 2 **Data:** expose the learner to relevant rare events through historical data, simulation, and careful reweighting.
- 3 **Evaluation:** report tails, constraint violations, and performance under distribution shift.

The RL algorithm is only one component: reliability comes from aligning the objective, the data, and the evaluation.

Case study: a flood-control policy is a full pipeline

Component	Design choice
State	reservoir, catchment, forecast distribution, season, barrier status
Action	continuous release plus discrete barrier decision
Dynamics	hydrological simulator calibrated on observed and generated extremes
Reward	water value, operating cost, disruption
Safety	overflow probability and flow constraints
Risk	CVaR of cumulative loss at a declared confidence level
Evaluation	nominal years, tail scenarios, shifted climate, model misspecification

The RL algorithm is one component; the state, simulator, objective, and evaluation define the decision system.

Open problems sit at the RL–extremes interface

- How can we learn tail-aware policies with few extreme trajectories?
- Which generative models preserve the dependence relevant for control?
- How should risk preferences adapt as evidence accumulates?
- Can we certify constraint satisfaction under model misspecification?
- How do multiple decision makers coordinate during cascading extremes?

This is a meeting point for RL, EVT, generative modeling, and control.

Five ideas to take away

- 1 RL optimizes **sequential decisions**, not predictions.
- 2 Bellman equations turn long-term consequences into local updates.
- 3 Model-free (Q-learning) and policy-gradient methods learn from sampled trajectories.
- 4 Expected return can conceal catastrophic tails and hard constraints.
- 5 For extreme events, train and evaluate on the **distribution of outcomes**.

Prediction tells us what may happen. RL asks what we should do about it.

Which part of your extreme-event model could become a decision problem?

State

What is known?

Action

What can change?

Risk criterion

Which outcomes are unacceptable?



Selected references



R. Sutton and A. Barto. *Reinforcement Learning: An Introduction*. 2nd ed., 2018.



V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 2015.



J. Schulman et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.



A. Tamar et al. Policy Gradient for Coherent Risk Measures. *NeurIPS*, 2015.



Y. Chow et al. A Lyapunov-based approach to safe reinforcement learning. *NeurIPS*, 2018.



S. Levine et al. Offline Reinforcement Learning: Tutorial, Review, and Perspectives. arXiv:2005.01643, 2020.



R. Agarwal et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS*, 2021.